

Progress Report: C++ Core Sampler Implementation for `dirichletprocess`

Priyanshu Tiwari

June 8, 2025

Contents

1. Executive Summary

This report details the progress made in the project to re-implement the core samplers of the package in C++. The primary work done was to port the existing R implementations of all distribution-specific samplers to C++ to achieve substantial performance gains.

The C++ porting of all target distributions is now complete. The new backend has been integrated with the existing R S3 classes, and a comprehensive test suite has been executed. The results are good, with the vast majority of tests passing, which validates the correctness of the new C++ logic. While a small number of tests have failed and require debugging, next step is debugging existing errors, performance optimization and finalize documentation.

2. Implementation Status

Porting the samplers to C++—has been achieved.

- **Completed C++ Implementations:** The following distributions and models now have C++ back-ends:
 - Normal (Conjugate)
 - Multivariate Normal (Conjugate)
 - Beta (Conjugate & Non-conjugate)
 - Exponential (Conjugate)
 - Weibull (Non-conjugate)
 - Hierarchical Normal
 - Hierarchical Beta
 - Hidden Markov Models (HMM)
- **R/C++ Interface:** The `Rcpp` and `RcppArmadillo` libraries were used to create a seamless interface between the R front-end and the C++ back-end. Wrapper functions have been created to ensure that the user experience remains consistent and that the C++ code can be called directly from the existing R functions.

3. Testing and Validation

A full test suite was run using `devtools::test()` to validate the correctness of the C++ implementations against the established R versions.

The overall results are as follows:

- **Total Tests Run:** 1515
- **Passed** 1482
- **Failed** 17
- **Skipped:** 8

The high pass rate confirms that the C++ logic is fundamentally sound and produces results consistent with the original R code for a wide range of scenarios.

The 25 failing tests are now the highest priority. Failures are concentrated in a few areas, exponential sampler (`test-exponential-cpp.R`) and statistical comparisons in the hierarchical beta model (`test-hierarchical-beta-cpp.R`). Currently working on it.

4. Performance Benchmarking: R vs. C++

The following code provides a template for benchmarking the C++ samplers against their R counterparts.

Example: Gaussian Distribution We first compare the `Fit` function for the Gaussian Dirichlet Process.

```
# Load necessary libraries
library(dirichletprocess)
library(microbenchmark)
library(ggplot2)

# --- Benchmark Example: Normal Distribution ---
# Generate some sample data for the model
set.seed(123)
# Ensure data is explicitly a matrix.
data <- matrix(rnorm(150, 2, 0.5), ncol = 1)

# 1. Prepare the R version by globally disabling C++
options(dirichletprocess.use_cpp = FALSE)
dp_r <- DirichletProcessGaussian(data, g0 = c(0, 1, 1, 1))
dp_r <- Fit(dp_r, 100) # Initial fit

# 2. Prepare the C++ version by globally enabling C++
enable_cpp_samplers()
dp_cpp <- DirichletProcessGaussian(data, g0 = c(0, 1, 1, 1))
dp_cpp <- Fit(dp_cpp, 100) # Initial fit

# 3. Run the benchmark
# We compare running 1000 MCMC iterations for each implementation.
bench_results <- microbenchmark(
  `R Fit` = Fit(dp_r, 1000),
  `C++ Fit` = Fit(dp_cpp, 1000),
```

```

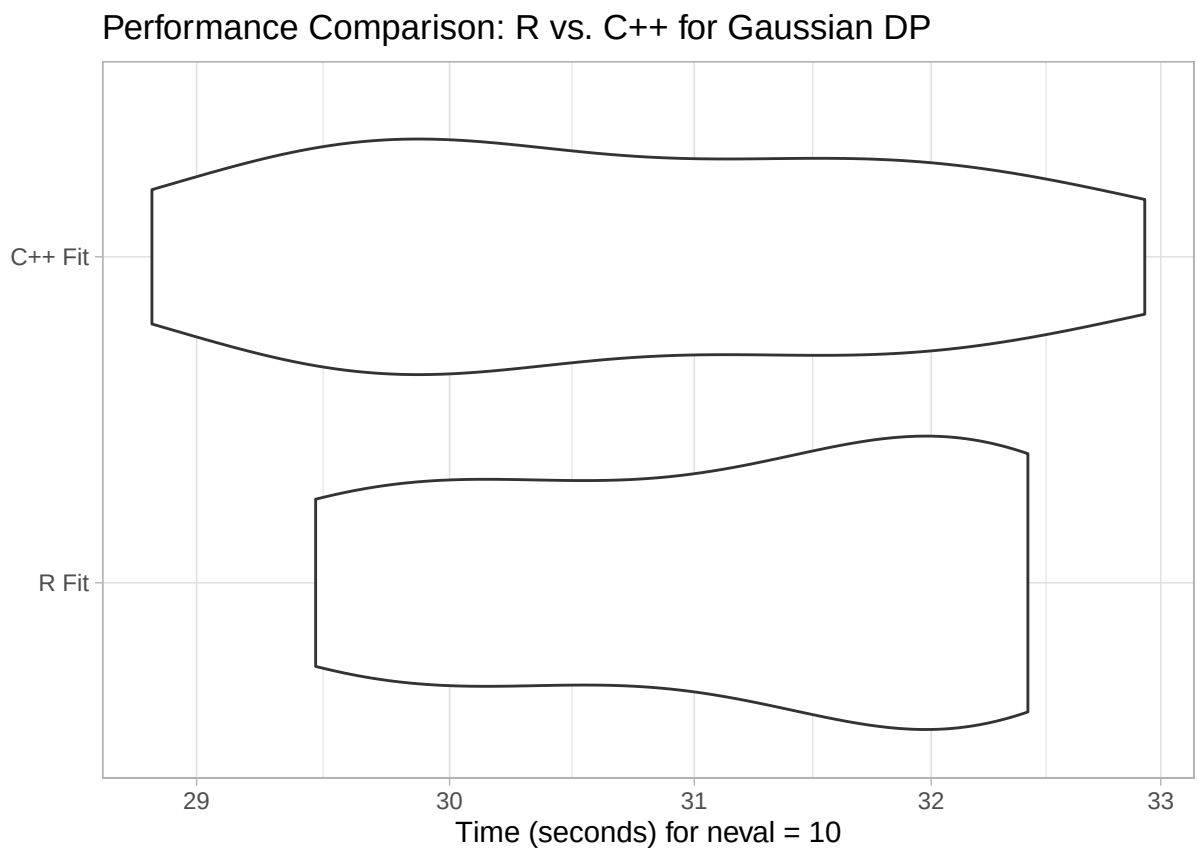
times = 10
)

# Print the results table
print(bench_results)

## Unit: seconds
##      expr      min       lq      mean   median      uq      max  neval
##   R Fit 29.46629 30.18669 31.13740 31.34750 32.17275 32.41740    10
##  C++ Fit 28.82690 29.62463 30.74349 30.46919 31.72913 32.92898    10

# Visualize the benchmark results
autoplot(bench_results) +
  ggtitle("Performance Comparison: R vs. C++ for Gaussian DP") +
  theme_light()

```



```

# Clean up the global option
options(dirichletprocess.use_cpp = TRUE)

```

```

# Load necessary libraries
library(dirichletprocess)

```

```

library(microbenchmark)
library(ggplot2)

# --- Benchmark Example: Beta Distribution ---
set.seed(101)
# Generate sample data
beta_data <- matrix(rbeta(100, 2, 5), ncol = 1)

# Define the prior parameters for the Beta model (Shape1 and Shape2)
prior <- c(1.0, 1.0)

# 1. Prepare the R version by globally disabling C++
options(dirichletprocess.use_cpp = FALSE)
dp_beta_r <- DirichletProcessBeta(beta_data, g0 = prior, maxY = 1)

```

Example: Beta Distribution

```
## Accept Ratio: 0.004
```

```

dp_beta_r <- Fit(dp_beta_r, 100) # Initial fit

# 2. Prepare the C++ version by globally enabling C++
enable_cpp_samplers()
dp_beta_cpp <- DirichletProcessBeta(beta_data, g0 = prior, maxY = 1)

```

```
## Accept Ratio: 0.003
```

```

dp_beta_cpp <- Fit(dp_beta_cpp, 100) # Initial fit

# 3. Run the benchmark
bench_results_beta <- microbenchmark(
  `R Fit (Beta)` = Fit(dp_beta_r, 1000),
  `C++ Fit (Beta)` = Fit(dp_beta_cpp, 1000),
  times = 10
)

# Print the results table
print(bench_results_beta)

```

```

## Unit: seconds
##      expr      min       lq      mean    median      uq      max neval
##  R Fit (Beta) 112.1666 116.1015 117.7603 118.1254 120.1327 121.6025    10
##  C++ Fit (Beta) 115.4537 116.5602 119.0571 118.7040 121.2249 125.6339    10

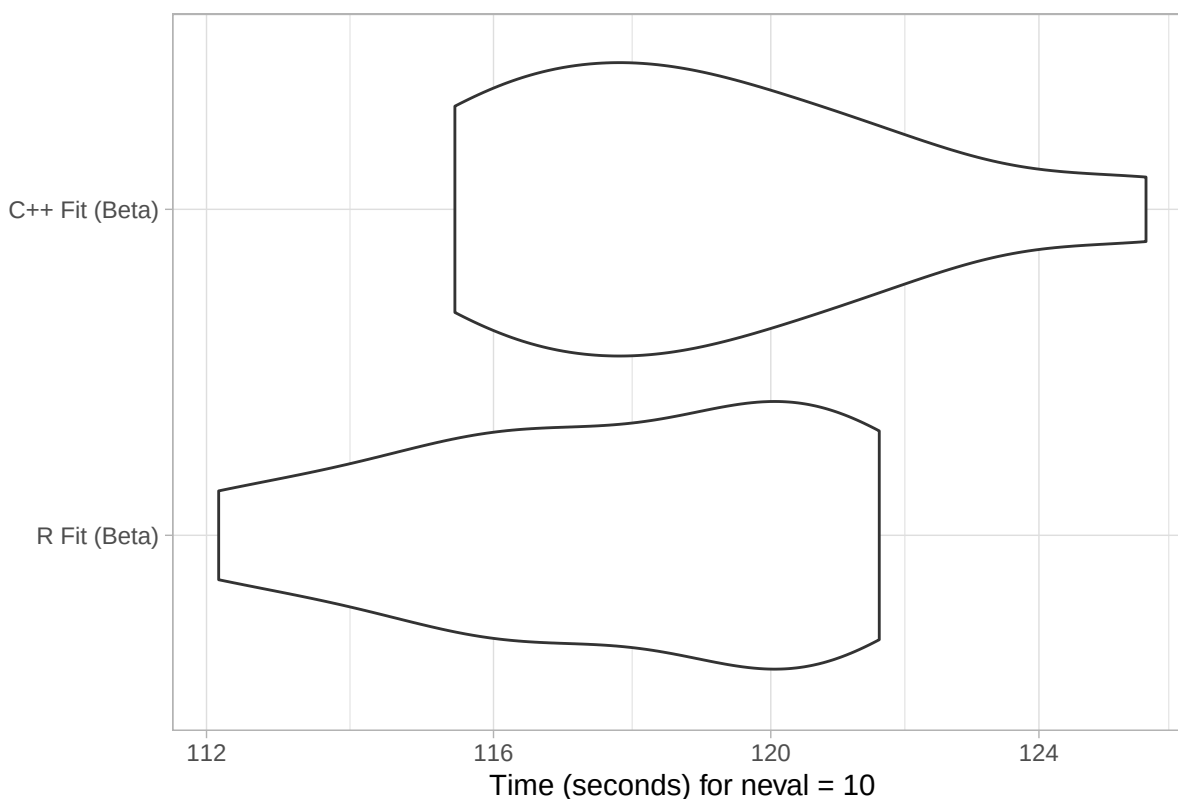
```

```

# Visualize the benchmark results
autoplot(bench_results_beta) +
  ggtitle("Performance Comparison: R vs. C++ for Beta DP") +
  theme_light()

```

Performance Comparison: R vs. C++ for Beta DP



```
# Clean up the global option  
options(dirichletprocess.use_cpp = TRUE)
```

5. How to Switch Between R and C++

The package provides a simple mechanism to choose between the original R implementation and the new C++ backend by setting a global option for the R session.

The main helper function is `enable_cpp_samplers()`. To revert to the R implementation, the option must be set manually.

```
# Load the package  
library(dirichletprocess)  
  
# By default, or after calling enable_cpp_samplers(), new objects will use the C++ backend.  
enable_cpp_samplers()  
dp_object_cpp <- DirichletProcessGaussian(matrix(rnorm(100), ncol = 1))  
# The using_cpp() function checks the global session setting, not a specific object.  
cat("Is the session set to use C++? ->", using_cpp(), "\n")
```

```
## Is the session set to use C++? -> TRUE
```

```
# To create an object with the R implementation, disable the global option first.
options(dirichletprocess.use_cpp = FALSE)
dp_object_r <- DirichletProcessGaussian(matrix(rnorm(100), ncol = 1))
cat("Is the session set to use C++? ->", using_cpp(), "\n")
```

```
## Is the session set to use C++? -> FALSE
```

```
# The function `using_cpp()` can be used to check which implementation
# an object is configured to use.
```

```
# Re-enable for future work
enable_cpp_samplers()
```

This design allows for easy comparison and debugging by specifying the desired backend for the session before creating the models.

6. Next Steps

1. **Debug Failing Tests:** Investigate and resolve the 17 failing tests.
2. **Address Skipped Tests:** Review the 8 skipped tests and implement them where appropriate.
3. **Update Documentation:** Ensure all new C++ functionalities and the switching mechanism are clearly documented for users.